

Reviewer Pack — Minimal Rank of Algebraic Curves Bounds Tree-like Resolution...

Entry f7d7d196e00e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 20:09:28 UTC

Minimal Rank of Algebraic Curves Bounds Tree-like Resolution Width

Entry ID: f7d7d196e00e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 20:09:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Algebraic Curves) **Field B** (complexity object): Boolean Function Complexity: Tree-like Resolution Proof Complexity

Statement:

For a given Boolean function f with n variables, the tree-like resolution width $t^*(f)$ satisfies the relation $O(\sqrt{n} \cdot \log n) \leq \min_{\{P \subseteq k^n\}} \text{rank}(A(P, f))$, where $A(P, f)$ is the matrix associated with the algebraic curve defined by the polynomial representation of f over the finite field with 2^k elements, and P is a set of points in k^n .

Rationale (proposer's reasoning):

Algebraic geometry provides a rich structure for studying Boolean functions through their algebraic representations. The rank of the matrices derived from these representations could potentially expose underlying complexity properties that are not evident through traditional computational approaches like tree-like resolution.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d0f3b71b0b347daf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all 30 random seed generations, the rank of the matrix $A(P, f)$ for each Boolean function f with n variables is bounded by $O(\sqrt{n} \cdot \log n)$, and no seed produces a rank exceeding this bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'algebraic curves' AND 'tree-like resolution width' AND 'resolution proof complexity' - 'Boolean function complexity' AND 'minimal rank' AND 'algebraic curve' - 'finite field' AND 'polynomial representation' AND 'resolution width'

Top relevant hits considered: - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e} -linear maps - [http://arxiv.org/abs/dg-ga/9708009v1] Tree-like curves and their inflection points - [http://arxiv.org/abs/1812.11710v1] Geometric Satake correspondence for affine Kac-Moody Lie algebras of type A - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1112.2313v2] QBF-Based Boolean Function Bi-Decomposition - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/1102.5335v1] Block Companion Singer Cycles, Primitive Recursive Vector Sequences, and Coprime Polynomial Pairs over Finite Fields - [http://arxiv.org/abs/2502.18759v1] Some permutation polynomials via linear translators

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        # Find pivot
        max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
        for j in range(i+1, m):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

    return A

def matrix_rank(A):
    rank = 0
    A_rref = gaussian_elimination(A)
    for row in A_rref:
        if any(row):
            rank += 1
```

```

        rank += 1
    return rank

def polynomial_representation(f, n):
    # Convert Boolean function to polynomial representation over GF(2^k)
    k = random.randint(3, 5) # Choose a random k between 3 and 5
    poly = [0] * (2**k - 1)
    for x in range(2**n):
        if f(x):
            poly[random.randint(0, len(poly)-1)] += 1
    return poly

def tree_like_resolution_width(f, n):
    # Placeholder function to compute tree-like resolution width
    # This is a dummy implementation and should be replaced with actual computation
    return random.randint(n//2, n)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        f = lambda x: random.choice([True, False]) # Placeholder Boolean function
        rank = matrix_rank(polynomial_representation(f, n))
        width = tree_like_resolution_width(f, n)

        results.append({
            "n": n,
            "rank": rank,
            "width": width
        })

    mean_rank = sum(result["rank"] for result in results) / len(results)
    conjecture_holds = all(result["rank"] <= math.sqrt(n) * math.log2(n) for result in results)
    counterexample = "" if conjecture_holds else f"Rank {result['rank']} exceeds bound for n={n}"

    return {
        "metric_name": "Rank of Matrix",
        "metric_value": mean_rank,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)] # Default to first 3 seeds

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Rank exceeds bound\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_436e6559.py", line 88, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_436e6559.py", line 61, in run_trial
    rank = matrix_rank(polynomial_representation(f, n))
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_436e6559.py", line 35, in matrix_rank
    A_rref = gaussian_elimination(A)
               ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_436e6559.py", line 19, in gaussian_elimination
    m, n = len(A), len(A[0])
              ~~~~~^
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support condition. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18614
2	preregistration	ollama_remote	glm4:latest	0	0	22592

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	16239
4	novelty	ollama_remote	glm4:latest	0	0	15896
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13022
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19568
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14047
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14332
9	judge	ollama_remote	glm4:latest	0	0	17184

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 151495 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/f7d7d196e00e.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/f7d7d196e00e.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/f7d7d196e00e.tar.gz* (if generated)